Tutorato 3

Matteo Mazzaretto

03 Novembre 2026



Template

Template

I template possono essere

1. Template di funzione
2. Template di classe

Assieme definiscono l'idea del cosiddetto *polimorfismo parametrico*.

Template di funzione

Sono descrizioni di un metodo che il compilatore può usare per generare automaticamente (in maniera implicita oppure esplicita) istanze particolari di una funzione che differiscono per il tipo degli argomenti.

```
template <class T>
T min(T a, T b) {
  return a < b ? a : b;
}
```

```
template <class T, int size>
bool min(T a) {
  return a < size;
}
```

Qui si presuppone che T possa essere confrontabile con *int*.

I parametri possono essere:

1. Parametri di tipo (preceduti da *class* o *typename*, possono essere di qualsiasi tipo)
2. Parametri valore (tipo predefinito)

Istanziazione

L’istanziazione può essere:

1. Implicita
2. Esplicita

Istanziazione implicita

Il tipo degli elementi viene dedotto da sinistra a destra, e nel caso in cui ci siano più parametri di tipo T questi devono essere **esattamente** uguali fra di loro.
Con l'esempio *min* precedente

```
int i; double d;
min(i, d); //ERRORE di compilazione perché sono due tipi diversi
```

Questo obbliga il programmatore a scegliere con cautela le istanziazioni implicite in quanto il compilatore non può sapere quale tipo veramente il programmatore voleva usare.
Tuttavia sono ammesse conversioni:

1. T& $\rightarrow$ T
2. T[] $\rightarrow$ T*
3. T $\rightarrow$ const T
4. T $\rightarrow$ const T&

Istanziazione esplicita

Riutilizzando l'esempio precedente, per fare compilare l'uso di due diversi tipi, si può fare così:

```
int i; double d;
min<double>(i, d); //compila, i diventa un double con la conversione int -> double
```

Solitamente si usa per risolvere ambiguità.

Uso di array statici

Quando si vuole usare un array statico, la dimensione costante dell'array è parte integrante del tipo del parametro, e per questo molto limitante all'uso di **qualsiasi** array statico. Per questo, quando si costruisce un template di funzione, si usa il parametro per **valore** *int size*.

```
template<class T, int size>
T min(T (&a)[size]) {
  ......
}

int main(){
  int ia[20];
  cout << min(ia);
  cout << min<int,20>(ia);
}
```

Compilazione

Compilazione dei template

La definizione del solo template non porta ad alcun oggetto compilabile.
Oltretutto, è impossibile “emulare” l’information hiding con i template, in quanto non essendo oggetti compilabili direttamente si è costretti a consegnare il template nei file header.
Tuttavia, c’è un modo per limitare la compilazione di (potenzialmente) infiniti template.

Compilazione per inclusione

Viene usato sia per i template di funzione che per i template di classe.
Le definizioni dei template vengono messe nei file header, i quali devono essere inclusi in ogni file che usa il template.
Tuttavia rimane il problema per cui se la stessa istanza di un template viene usata in più file compilati separamente il codice per tale istanza viene generato più volte dal compilatore.
Si può forzare il compilatore a generare le istanze del template che vengono usate nel programma con delle *dichiarazioni esplicite di istanziazione*.

```
template <class T>
T min(T a, T b) { return a < b ? a : b; }

template int min(int, int);
```

La flag **-fno-implicit-templates** comunica che una dichiarazione esplicita di istanziazione compare in un altro file e che il template di funzione non deve essere istanziato.

Template di classe

Come funziona

È la descrizione di un metodo che il compilatore può usare per generare automaticamente istanze particolari di una classe che differiscono per il tipo di alcuni membri, l'istanziazione è sempre **esplicita**

```
template<class T>
class Queue{
  public:
    Queue();
    ~Queue();
    bool is_empty() const;
    void add(const T&);
    T remove();
};
```

Per definire una classe Queue (coda)

```
Queue<int> qi;
```

I parametri, sia di tipo che di valore, possono avere parametri di default.

Queue e QueueItem

```
template <class T>
class QueueItem;

template <class T>
class Queue{
  public:
    Queue();
    ~Queue();
    bool is_empty() const;
    void add(const T&);
    T remove();
  private:
    QueueItem<T>* primo;
    QueueItem<T>* ultimo;
};
```

```
template <class T>
class QueueItem{
  public:
    QueueItem(const T&);
    T info;
    QueueItem* next;
};
```

Istanziazioni di un template di classe

Nel caso in cui vengano generate più classi, ad esempio

- Queue<int>
- Queue<string>

Sono due classi completamente distinte che non possono accedere fra di loro alla parte privata della classe.

```
template<class T>
int f(Queue<T>& qT, Queue<string> qS);
```

È completamente legale.
Anche in questo caso, per definire template di classe o funzione si può sfruttare una dichiarazione incompleta della classe.

Definizione esterna

```
template<class T>
class Queue{
  public:
    Queue();
  private:
    QueueItem<T>* primo;
    QueueItem<T>* ultimo;
};

//definizione esterna
template <class T>
Queue<T>::Queue() : primo(0), ultimo (0) {}
```

Il metodo di un template di classe non viene istanziato quando viene istanziata la classe ma se e soltanto quando il programma usa effettivamente quel metodo.

Dichiarazioni friend

Possibili dichiarazioni

1. Dichiarazione nel template di classe C di una classe o funzione friend non template
2. Dichiarazione nel template di classe C di un template di classe o di un template di funzione friend associato, cioè avente tra i suoi parametri alcuni dei parametri di template C
3. Dichiarazione nel template di classe C di un template di classe o di un template di funzione friend non associato, cioè avente i parametri disgiunti dai parametri di C

Esempio 1

```
class A {
  ...
  int fun();
  ...
};

template<class T>
class C{
  friend int A::fun();
  friend class B;
  friend bool test();
};
```

La classe B, la funzione test() e il metodo A::fun() sono friend di **tutte le istanze** del template di classe C.

Esempio 2

```
template <class T> class A{... int fun(); ...};
template <class T> class B { ... };

template<class T1, class T2> class C;

template<class T1, class T2> bool test(C<T1,T2>);

template<class T1, class T2> class C{
  friend int A<T1>::fun();
  friend class B<T2>;
  friend bool test<T1,T2>(C);
};
```

Ad ogni istanza del template di classe C rimane associata come amica **una ed una sola** istanza del template di classe B e dei template di funzione A<T>::fun() e test.
Le dichiarazioni dei template di classe A e B e del template di funzione test devono essere visibili quando viene definito il template di classe C.
Per dichiarare test è necessaria la dichiarazione (anche incompleta) di C.

Esempio 3

```
template <class T>
class C{
  T t;
  template<class Tp>
  friend int A<Tp>::fun();

  template <class Tp>
  friend class B;

  template<class Tp>
  friend bool test(C<Tp>);
};
```

I template di classe o di funzione dichiarati friend sono friend di ogni istanza del template di classe C.

Membri statici

Se un template di classe contiene un membro statico, ne viene creato uno nuovo per ogni nuova istanza della classe.

```
template<int I>
class C{
  static int numero;
  public:
    C();
};

template<int I>
int C<I>::numero = I;

template<int I>
C<I>::C() { numero ++; }

int main(){
  C<1> uno; C<2> due_a; C<2> due_b;
}
```

Typename

Typename

La parola chiave *typename* ha due usi:

1. Come **alternativa a *class*** nella lista dei parametri di tipo
2. Per indicare che un **nome dipendente** è un tipo

```
template <class T>
T min(T a, T b);

template <typename T>
T min(T a, T b);
```

Nomi dipendenti

Dentro un template, un nome qualificato che dipende da un parametro (come `T::x`) non è risolvibile finché T non è noto. Per default il compilatore assume che sia un **valore** (es. un membro statico), non un tipo.

```
template <class T>
void f() {
  T::x * p;
  // dichiarazione di un puntatore p a T::x
  // oppure moltiplicazione tra T::x e p?
}
```

Il compilatore sceglie la moltiplicazione. Per dire che `T::x` è un tipo si scrive

```
typename T::x * p;
```

Quando è obbligatorio

È necessario davanti a un nome **qualificato** e **dipendente** da un parametro di template che denota un tipo.

```
template <class T>
void stampa(const T& c) {
  typename T::const_iterator it = c.begin();
  for (; it != c.end(); ++it)
    cout << *it << endl;
}
```

Non si usa:

- Per nomi non dipendenti (es. `std::string`)
- Nella lista delle classi base e negli inizializzatori del costruttore

Contenitori STL

Standard Template Library

È una libreria di contenitori.
Ogni funzione indica la complessità massima con la quale deve essere presente.
STL ha contenitori, iteratori, algoritmi.
Chiaramente sono ridefiniti tutti gli operatori importanti (la regola del 3) e gli operatori di confronto (<, $\leq$ ecc).

Vector

Restituisce array dinamico con capacity()≥size().
È il più semplice contenitore e per la maggioranza delle applicazioni è il più efficiente.

1. Supporto accesso casuale O(1)
2. Inserimento e rimozione **in coda** a tempo costante ammortizzato
3. Inserimento e rimozione **arbitraria** a tempo lineare ammortizzato
4. La capacità può variare dinamicamente (potenza di 2)
5. La gestione della memoria è automatica

La capacità raddoppia ogni volta che c'è bisogno di una cella oltre capacity().

Template vector

Template a 2 parametri:

```
template <class T, class Alloc = allocator<T>> class vector;
```

Il primo è il tipo degli elementi, il secondo l'allocatore (quasi sempre quello di default).
Size iniziale e valore di default si passano al **costruttore**:

```
vector<int> v;          // vuoto
vector<int> v(10);      // 10 elementi, tutti 0
vector<int> v(10, 7);   // 10 elementi, tutti 7
```

Metodi fondamentali

```
void push_back(const T&); //aggiungi in coda
void pop_back(); //rimuovi in coda
T& front();
T& back();
iterator begin();
iterator end();
bool empty(); //comune a tutti i contenitori
int size(); //comune a tutti i contenitori
```

Ad ogni classe contenitore si ha

- C::iterator
  - permette di iterare sul contenitore e modificare gli elementi puntati
- C::const_iterator
  - permette di iterare sul contenitore ma **non** di modificare gli elementi puntati (da non confondere con const const_iterator che non permette né di iterare né di modificare gli elementi puntati)

Iteratori

Un iteratore si comporta come un puntatore: `*it` accede all'elemento, `++it` passa al successivo.

```
vector<int> v;
v.push_back(1); v.push_back(2);

for (vector<int>::iterator it = v.begin(); it != v.end(); ++it)
  cout << *it << endl;
```

Dentro un template serve `typename`:

```
template <class T>
void stampa(const vector<T>& v) {
  typename vector<T>::const_iterator it = v.begin();
  ...
}
```

Altri contenitori

Esistono altri contenitori come:

- list (lista doppiamente linkata)
- deque (coda con accessi sia dal “fronte” che dal “retro”)